

```
In [45]: ## Importing the Data
```

```
In [33]: #importing the necessary packages.
import pandas as pd
import numpy as np
import seaborn as sb
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
```

```
In [6]: #importing the dataset in excel format.
data = pd.read_excel('DataAnalyst_Assesment_Dataset.xlsx', header = 0)
```

```
In [7]: #displaying the first 5 rows of the table.
data.head()
```

```
Out[7]:
```

	Booking ID	Customer ID	Customer Name	Booking Type	Booking Date	Status	Class Type	Instructor
0	279d92c6-ce26-47c0-8915-e45b77fe20e2	00901ce3-3d86-4c97-bca2-40ccac2fb99f	Customer 1	Facility	2025-05-30	Pending	NaN	NaN
1	415bfcbe-1a2e-4d4b-809a-4c5b606653b1	b82db986-bd52-4b07-bdd8-aa8cf2016241	Customer 2	Birthday Party	2025-05-29	Pending	NaN	NaN
2	2100024b-46fc-47b5-ac1c-047d007a4723	6bbb6e83-9577-4f64-80b0-f073132d18f3	Customer 3	Birthday Party	2025-05-09	Confirmed	NaN	NaN
3	74936def-088f-4d34-bad1-dfa76f78b704	f16f5beb-6a7d-4493-a19e-a30dbbd206e9	Customer 4	Birthday Party	2025-06-07	Pending	NaN	NaN
4	6272b4e7-a508-4ed7-bae0-21f7293287a8	eb297435-93d1-4e65-8dd4-6450922305cb	Customer 5	Class	2025-04-13	Pending	Art	NaN

```
In [46]: # CLEANING & MANUPULATING THE DATA
```

```
In [8]: #identifying the missing values
missing_values = data.isnull().sum() ##using isnull() and sum() to identify missing
print("Missing Values:", missing_values)
```

Missing Values: Booking ID	0
Customer ID	0
Customer Name	0
Booking Type	0
Booking Date	0
Status	0
Class Type	672
Instructor	730
Time Slot	205
Duration (mins)	205
Price	0
Facility	328
Theme	727
Subscription Type	1000
Service Name	0
Service Type	0
Customer Email	94
Customer Phone	105

dtype: int64

```
In [29]: #Imputing the missing values using SimpleImputer() funntion.
imputer1 = SimpleImputer(strategy='most_frequent') ##using most_frequent strategy f
imputer1.fit(data[['Time Slot']])
data[['Time Slot']] = imputer1.transform(data[['Time Slot']])
```

```
In [20]: imputer2 = SimpleImputer(strategy = 'most_frequent')
imputer2.fit(data[['Facility']])
data[['Facility']] = imputer2.transform(data[['Facility']])
```

```
In [21]: imputer3 = SimpleImputer(strategy = 'most_frequent')
imputer3.fit(data[['Class Type']])
data[['Class Type']] = imputer3.transform(data[['Class Type']])
```

```
In [22]: imputer4 = SimpleImputer(strategy = 'most_frequent')
imputer4.fit(data[['Instructor']])
data[['Instructor']] = imputer4.transform(data[['Instructor']])
```

```
In [23]: imputer5 = SimpleImputer(strategy = 'most_frequent')
imputer5.fit(data[['Duration (mins)']])
data[['Duration (mins)']] = imputer5.transform(data[['Duration (mins)']])
```

```
In [24]: imputer6 = SimpleImputer(strategy = 'most_frequent')
imputer6.fit(data[['Theme']])
data[['Theme']] = imputer6.transform(data[['Theme']])
```

```
In [30]: #checking for missing values again.
missing_values = data.isnull().sum()
print("Missing Values:", missing_values)
```

```

Missing Values: Booking ID      0
Customer ID                    0
Customer Name                  0
Booking Type                   0
Booking Date                   0
Status                         0
Class Type                     0
Instructor                     0
Time Slot                      0
Duration (mins)                0
Price                          0
Facility                       0
Theme                          0
Subscription Type              1000
Service Name                   0
Service Type                   0
Customer Email                 94
Customer Phone                 105
dtype: int64

```

```

In [31]: #identifying the datatypes.
         data.dtypes

```

```

Out[31]: Booking ID           object
         Customer ID          object
         Customer Name         object
         Booking Type          object
         Booking Date          datetime64[ns]
         Status                object
         Class Type            object
         Instructor            object
         Time Slot             object
         Duration (mins)       float64
         Price                 float64
         Facility              object
         Theme                 object
         Subscription Type     float64
         Service Name          object
         Service Type          object
         Customer Email        object
         Customer Phone        object
         dtype: object

```

```

In [52]: #Dropping the unrequired column as it not having any values.
         data = data.drop(columns=['Subscription Type'])

```

```

In [34]: #assigning correct datatypes.
         data['Time Slot'] = data['Time Slot'].apply(lambda t: t.hour)

```

```

In [59]: #fixing incorrect values
         unique_values = data['Price'].unique()
         unique_values

```

```
Out[59]: array([ 42.74, 182.06, 207.5 , 203.2 , 161.14, 278.07, 161.82, 282.96,
  0.    , 44.41, 47.55, 186.42, 198.3 , 83.48, 180.94, 30.53,
 173.37, 150.79, 162.42, 116.   , 135.43, 120.73, 74.15, 224.56,
 298.54, 276.61, 18.07, 90.07, 225.96, 215.14, 296.42, 296.34,
 19.06, 49.57, 174.04, 136.25, 43.11, 187.17, 261.97, 254.36,
 192.87, 246.09, 180.88, 56.44, 98.11, 57.54, 299.05, 69.58,
 70.18, 195.14, 257.25, 90.65, 98.84, 272.82, 280.1 , 73.87,
 260.1 , 52.33, 228.65, 124.08, 160.77, 138.21, 170.44, 231.64,
 37.12, 18.51, 127.86, 299.76, 243.4 , 117.62, 240.83, 205.89,
 76.26, 192.91, 36.74, 213.13, 82.11, 146.2 , 298.04, 130.82,
 47.49, 43.19, 125.53, 277.9 , 197.21, 18.58, 239.06, 247.36,
 240.62, 185.46, 144.91, 297.06, 190.03, 286.08, 286.6 , 213.84,
 80.9 , 64.28, 33.89, 121.26, 176.07, 71.98, 19.87, 69.56,
 104.25, 227.   , 278.42, 278.4 , 249.19, 184.39, 166.99, 53.49,
 176.35, 244.52, 78.85, 122.05, 87.48, 23.8 , 212.28, 95.59,
 190.31, 131.05, 35.19, 209.84, 231.49, 150.5 , 34.98, 103.14,
 64.25, 285.35, 110.77, 22.61, 294.43, 195.84, 283.81, 97.41,
 141.96, 107.02, 289.29, 148.73, 296.6 , 162.76, 16.93, 264.74,
 40.49, 33.24, 201.89, 230.21, 235.17, 244.18, 56.25, 174.09,
 211.3 , 220.18, 55.33, 137.36, 60.88, 107.08, 96.69, 149.01,
 281.4 , 266.54, 279.47, 20.36, 113.24, 182.1 , 24.28, 172.52,
 183.95, 47.02, 183.34, 295.4 , 170.86, 131.73, 113.2 , 249.36,
 126.05, 122.4 , 165.16, 228.87, 38.67, 219.84, 32.4 , 134.37,
 84.7 , 34.89, 110.9 , 189.78, 211.26, 245.31, 146.02, 196.32,
 50.34, 225.49, 199.16, 191.75, 276.31, 215.74, 216.99, 262.05,
 253.63, 205.4 , 87.29, 105.56, 150.25, 141.88, 165.66, 40.06,
 224.36, 234.3 , 32.15, 117.69, 31.74, 167.42, 79.23, 224.86,
 81.3 , 292.66, 124.41, 258.61, 28.95, 112.03, 100.24, 263.4 ,
 176.4 , 222.77, 197.9 , 88.13, 274.8 , 229.87, 288.79, 77.3 ,
 35.53, 138.98, 267.21, 148.18, 287.54, 227.12, 103.89, 86.66,
 262.73, 53.1 , 53.82, 220.33, 79.12, 169.19, 246.07, 30.49,
 151.81, 258.64, 239.39, 15.25, 144.52, 293.64, 133.09, 242.16,
 15.67, 247.88, 124.76, 99.44, 179.6 , 248.83, 269.23, 275.82,
 84.35, 210.23, 79.6 , 237.06, 138.61, 135.28, 179.11, 28.44,
 225.62, 29.8 , 229.08, 230.31, 28.97, 283.34, 70.5 , 287.16,
 52.98, 253.5 , 272.8 , 52.03, 194.89, 196.85, 134.04, 152.36,
 91.99, 211.24, 299.14, 130.8 , 40.73, 51.83, 35.07, 208.45,
 60.68, 222.94, 207.04, 224.27, 164.35, 148.46, 196.43, 207.27,
 94.45, 285.76, 72.47, 246.19, 251.13, 270.23, 291.39, 117.22,
 178.63, 231.74, 212.07, 197.86, 61.41, 18.38, 163.45, 291.48,
 127.87, 137.92, 92.17, 253.29, 15.37, 174.52, 285.94, 279.   ,
 281.23, 130.9 , 94.25, 155.98, 43.4 , 119.4 , 95.01, 64.1 ,
 137.35, 63.09, 104.97, 263.96, 219.01, 298.75, 236.75, 210.   ,
 29.23, 123.63, 173.59, 194.58, 120.34, 151.25, 210.05, 209.03,
 219.38, 121.24, 222.13, 115.62, 222.25, 171.43, 271.3 , 131.92,
 37.57, 234.66, 21.13, 47.22, 83.96, 220.07, 127.4 , 45.55,
 141.81, 62.43, 33.17, 171.93, 34.55, 119.11, 282.11, 232.57,
 163.51, 222.33, 229.51, 233.04, 99.4 , 73.7 , 124.9 , 127.24,
 205.37, 292.93, 177.27, 124.19, 235.99, 181.88, 79.95, 133.21,
 223.02, 100.9 , 139.31, 141.01, 218.14, 103.42, 238.6 , 214.69,
 168.68, 24.75, 70.74, 120.99, 106.41, 110.21, 61.72, 101.01,
 230.35, 138.36, 36.22, 293.68, 89.88, 142.51, 118.47, 126.31,
 201.67, 292.3 , 128.83, 85.85, 223.99, 248.03, 101.66, 47.35,
 122.68, 115.8 , 114.38, 38.21, 170.45, 49.21, 91.89, 183.19,
 96.12, 46.53, 216.72, 243.28, 280.59, 297.19, 120.8 , 188.54,
 167.47, 40.33, 47.15, 157.42, 147.24, 84.45, 27.61, 200.33,
```

72.99, 39.33, 50.9 , 208.08, 44.53, 277.32, 104.36, 179.4 ,
82.22, 133.33, 25.01, 299.29, 149.45, 83.29, 247.12, 191.35,
298.15, 193.89, 45.27, 53.03, 107.77, 126.5 , 270.74, 258.04,
265.74, 101.95, 26.88, 104.9 , 242.2 , 112.68, 60.74, 108.42,
193.34, 206.43, 223.93, 90.53, 223.95, 82.41, 290.64, 286.15,
111.92, 139.42, 296.11, 120.15, 100.81, 49.04, 64.42, 198.64,
119.82, 170.84, 29.43, 105.21, 174.95, 270.87, 289.91, 32.66,
147.22, 244.02, 284.6 , 199.78, 170.43, 126.63, 293.32, 59.72,
75.75, 282.86, 60.14, 143.2 , 88.64, 61.35, 172.12, 120.52,
30.09, 165.26, 190.18, 231.46, 43.35, 217.57, 187.45, 166.7 ,
114.71, 173.89, 128.21, 181.17, 162.94, 123.26, 76.87, 93.52,
123.64, 97.12, 182.36, 296.21, 264.69, 262.94, 92.28, 39.65,
297.45, 24.06, 251.42, 114.36, 274.56, 94.39, 189.31, 270.04,
105.06, 61.95, 202.84, 262.07, 94.29, 231.89, 19.34, 130. ,
81.23, 58.46, 155.21, 42.07, 136.07, 93.74, 210.95, 219.58,
290.84, 265.22, 116.12, 279.73, 52.57, 294.41, 16.75, 250.93,
285.37, 117.46, 284.27, 227.14, 61.07, 170.2 , 243.93, 94.18,
83.05, 227.56, 248.91, 159.06, 66.21, 216.06, 153.96, 273.38,
246.26, 236.16, 237.49, 71.92, 182.29, 40.2 , 286.13, 19.84,
89.59, 146.01, 22.12, 228.47, 270.69, 276.5 , 69.59, 188.64,
36.04, 155.68, 159.03, 125.66, 146.73, 161.47, 115.46, 198.2 ,
233.66, 270.98, 299.62, 198.84, 210.87, 281.93, 97.83, 17.05,
111. , 282.85, 53.25, 53.81, 156.88, 177.1 , 238.94, 26.21,
32.71, 172.56, 53.33, 86.88, 76.99, 146.07, 269.9 , 196.94,
187.59, 204.36, 272.61, 227.39, 172.33, 187.78, 272.54, 108.54,
280.96, 256.82, 66.6 , 226.55, 181.61, 249.27, 163.9 , 98.23,
40.31, 99.43, 255.41, 230.93, 171.64, 186.97, 246.44, 56.57,
256.87, 20.89, 259.07, 91.59, 231.81, 206.15, 251.44, 52.58,
84.13, 48.72, 91.56, 100.68, 167.67, 116.33, 183.23, 152.35,
192.93, 258.81, 252.63, 207.74, 261.75, 86.3 , 141.68, 133.94,
54.46, 93.57, 250.67, 288.08, 263.3 , 187.52, 243.43, 48.62,
69.73, 30.32, 18.24, 24.54, 146.17, 250.13, 57.85, 283.64,
170.31, 195.41, 56.4 , 255.82, 63.07, 233.02, 83.37, 123.7 ,
69.02, 144.5 , 97.61, 279.72, 133.79, 19.13, 263.92, 213.08,
286.03, 185.23, 110.5 , 199.96, 70.85, 218.39, 186.55, 137.88,
39.93, 72.34, 50.45, 35.92, 84.4 , 239.23, 182.81, 116.03,
218.28, 59.2 , 58.17, 94.05, 221.16, 102.21, 85.43, 217.64,
42.38, 15.41, 256.29, 261.73, 168.73, 240.71, 188.51, 113.98,
16.47, 288.94, 273.2 , 83.97, 100.19, 225.74, 256.32, 104.42,
121.32, 170.9 , 112.63, 181.06, 290.35, 93.48, 97.18, 191.65,
178.96, 33.28, 197.37, 63.14, 115.49, 274.31, 39.77, 88.54,
231.7 , 78.27, 258.03, 202.46, 145.66, 191.91, 249.89, 205.06,
195.16, 172.97, 154.85, 123.37, 23.23, 66.53, 68.46, 94.83,
254.85, 276.01, 287.39, 159.42, 92.46, 231.6 , 289.38, 78.97,
199.89, 215.15, 104.74, 268.95, 262.54, 100.97, 288.29, 16.28,
220.88, 158.95, 231.95, 221.58, 92.62, 38.46, 139.44, 123.28,
195.08, 260.95, 22.83, 214.61, 227.32, 241.78, 148.16, 150.6 ,
251.6 , 206.74, 244.43, 222.99, 272.32, 251.72, 23.66, 161.45,
182.74, 210.02, 238.3 , 104.41, 147.94, 49.37, 143.65, 90.76,
206.71, 115.3 , 145.36, 116.07, 146.34, 45.13, 174.5 , 179.61,
178.74, 51.13, 21.09, 60.81, 295.57, 88.88, 165.84, 207.71,
290.03, 43.79, 191.23, 180.62, 37.31, 188.63, 63.87, 102.88,
105.4 , 179.95, 212.87, 231.22, 236.52, 120.1 , 141.7 , 104.83,
119.02, 172.84, 275.18, 130.08])

```
In [60]: #replacing zeros with the median of the Price column
data['Price'] = data['Price'].replace(0, data['Price'].median())
```

```
In [54]: data.dtypes
```

```
Out[54]: Booking ID          object
Customer ID          object
Customer Name        object
Booking Type         object
Booking Date         datetime64[ns]
Status               object
Class Type           object
Instructor           object
Time Slot            int64
Duration (mins)      float64
Price                float64
Facility             object
Theme                object
Service Name         object
Service Type         object
Customer Email       object
Customer Phone       object
dtype: object
```

```
In [55]: #identifying duplicate records in tables
duplicates = data.duplicated()
data[duplicates]
```

```
Out[55]:
```

Booking ID	Customer ID	Customer Name	Booking Type	Booking Date	Status	Class Type	Instructor	Time Slot	Duration (m)
------------	-------------	---------------	--------------	--------------	--------	------------	------------	-----------	--------------

```
In [56]: #sorting the data
sorted_data = data.sort_values(by = 'Booking Date', ascending = True)
print(sorted_data)
```

	Booking ID \
40	faf970c8-b0f4-40d4-9a8d-6d73abd5240c
243	2bb844a9-2b47-4ac5-adde-8c9d5491de9f
749	ab8029cd-420a-47c1-a441-4354c9ee6622
622	15104fe5-4348-4126-85e9-2fee1a636599
623	090ba787-bcd9-4b14-bdae-81aec5736e3b
..	...
298	c9848fd1-e38c-4f85-acc5-0e03ce0a56a6
386	38e8f4c0-60a8-4b6b-a426-d53bd0e03a18
414	450ba31b-7b40-4d76-8fc7-3f2b5d447c1c
404	10f6c1ff-eaf6-4e08-bb35-2a17dda0e74b
140	3673c31c-a475-451d-8845-e292e0caad99

	Customer ID	Customer Name	Booking Type \
40	258e2aa6-253e-4b5f-a1b4-1922550881e0	Customer 41	Facility
243	f3d1e339-241a-427d-822f-a69598a7588a	Customer 244	Birthday Party
749	dfb87234-793b-4500-b67d-e9fc7c4973cf	Customer 750	Birthday Party
622	1179f475-dd0e-43bc-9881-313e56e20912	Customer 623	Facility
623	d4568770-adc7-46d4-981a-41f7061bea0b	Customer 624	Facility
..	...	...	...
298	c999e371-086b-4bcf-934b-6c1e0c5e4bae	Customer 299	Birthday Party
386	f27d0844-6778-4476-b12e-446d14640928	Customer 387	Birthday Party
414	17ee5b69-4e73-450d-97dc-5c1e7406ab7c	Customer 415	Birthday Party
404	e195020c-df75-4f3b-b1cc-9aa8a7413f3c	Customer 405	Facility
140	dd14a25d-9648-40fd-b007-f460387725d1	Customer 141	Facility

	Booking Date	Status	Class Type	Instructor	Time Slot \
40	2025-03-25	Pending	Gymnastics	Amanda Davis	9
243	2025-03-25	Pending	Gymnastics	Amanda Davis	10
749	2025-03-25	Pending	Gymnastics	Amanda Davis	14
622	2025-03-25	Pending	Gymnastics	Amanda Davis	12
623	2025-03-25	Confirmed	Gymnastics	Amanda Davis	16
..	...	...	...	...	...
298	2025-06-22	Pending	Gymnastics	Amanda Davis	14
386	2025-06-22	Confirmed	Gymnastics	Amanda Davis	12
414	2025-06-22	Confirmed	Gymnastics	Amanda Davis	12
404	2025-06-22	Confirmed	Gymnastics	Amanda Davis	10
140	2025-06-22	Pending	Gymnastics	Amanda Davis	14

	Duration (mins)	Price	Facility	Theme	Service Name \
40	120.0	43.11	Play Area	Superhero	Play Area
243	45.0	124.41	Party Room	Superhero	Party Room
749	90.0	255.41	Play Area	Sports	Play Area
622	45.0	262.94	Party Room	Superhero	Party Room
623	45.0	92.28	Play Area	Superhero	Play Area
..	...	...	...	...	...
298	90.0	79.60	Party Room	Superhero	Party Room
386	90.0	123.63	Party Room	Sports	Party Room
414	90.0	141.81	Play Area	Superhero	Play Area
404	90.0	271.30	Party Room	Superhero	Party Room
140	90.0	34.98	Play Area	Superhero	Play Area

	Service Type	Customer Email	Customer Phone
40	Facility	customer41@example.com	001-741-9047
243	Birthday Party	customer244@example.com	001-588-4229
749	Class	customer750@example.com	001-939-7549

622	Facility	customer623@example.com	001-515-3584
623	Facility	customer624@example.com	001-263-2010
..	...	...	...
298	Birthday Party	customer299@example.com	001-331-6862
386	Birthday Party	customer387@example.com	001-309-8509
414	Birthday Party	customer415@example.com	001-821-8282
404	Facility	customer405@example.com	001-212-6926
140	Birthday Party	customer141@example.com	001-787-8904

[1000 rows x 17 columns]

In [47]: `# EXPLORATORY DATA ANALYSIS`

In [42]: `#identifying the frequency of booking types customers used`
`freq_booking = data['Booking Type'].value_counts()`
`freq_booking`

Out[42]: Booking Type
Facility 337
Birthday Party 335
Class 328
Name: count, dtype: int64

In [49]: `#finding the percentage of booking types customers used`
`perc_booking = freq_booking/len(data['Booking Type'])*100`
`perc_booking`

Out[49]: Booking Type
Facility 33.7
Birthday Party 33.5
Class 32.8
Name: count, dtype: float64

In [43]: `#identifying the number of bookings in confirmed state`
`freq_status = data['Status'].value_counts()`
`freq_status`

Out[43]: Status
Pending 510
Confirmed 490
Name: count, dtype: int64

In [61]: `#descriptive analysis by rounding the data`
`round(data.describe())`

Out[61]:

	Booking Date	Time Slot	Duration (mins)	Price
count	1000	1000.0	1000.0	1000.0
mean	2025-05-07 17:08:09.600000256	13.0	86.0	155.0
min	2025-03-25 00:00:00	9.0	45.0	15.0
25%	2025-04-14 00:00:00	11.0	45.0	97.0
50%	2025-05-08 00:00:00	14.0	90.0	137.0
75%	2025-05-31 00:00:00	14.0	90.0	220.0
max	2025-06-22 00:00:00	17.0	120.0	300.0
std	NaN	2.0	27.0	78.0

```
In [63]: #analysis by group by method
service_type_amount = data.groupby('Service Type')['Price'].median()
service_type_amount
```

Out[63]: Service Type
Birthday Party 137.355
Class 138.170
Facility 138.210
Name: Price, dtype: float64

```
In [68]: #creating a pivot table to analyze the total duration of classes by Service Type an
pivot_table = data.pivot_table(
    index='Service Type',      # rows
    columns='Facility',        # column
    values='Duration (mins)',  # values
    aggfunc='sum',             # agg function
    fill_value=0               # to replace NaN with 0
)
pivot_table
```

Out[68]:

	Facility	Party Room	Play Area
Service Type			
Birthday Party	11730.0	16155.0	
Class	4725.0	23310.0	
Facility	10800.0	18840.0	

```
In [73]: #saving the cleaned data in an exxel format
data.to_excel('cleaned_data.xlsx', index=False)
```

```
In [ ]:
```